

# View-Curriculum-GS: Pose-Similarity View Scheduling for 50-Second 3D Gaussian Splatting

Anonymous Authors

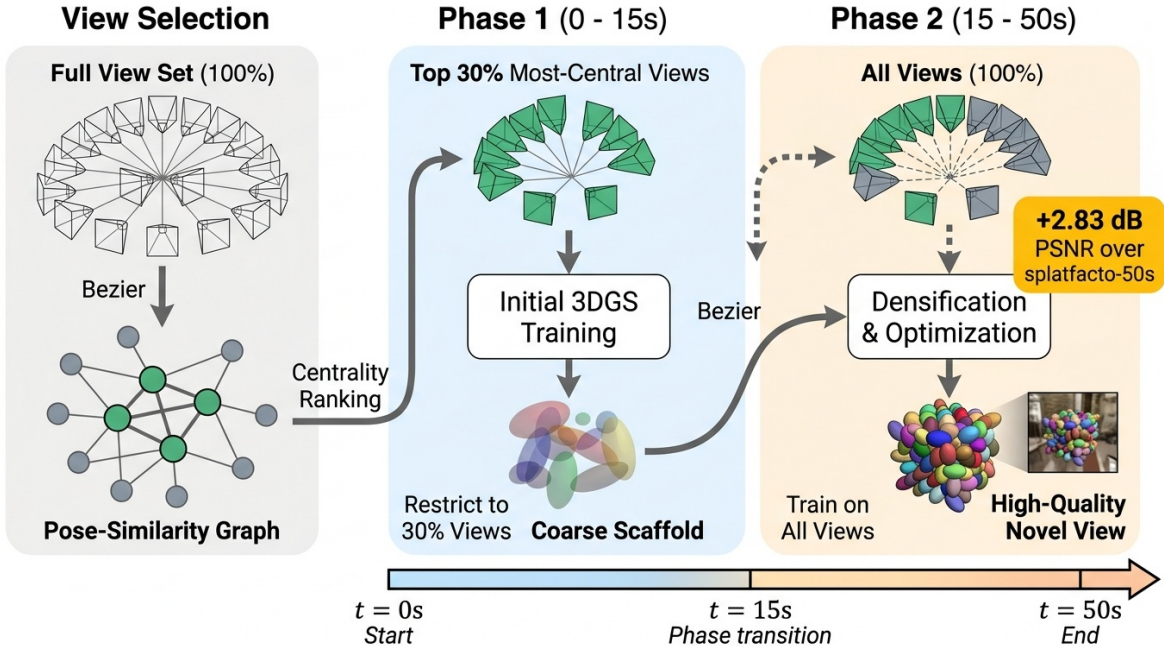


Figure 1: **View-Curriculum-GS**. Training on only the 30% most pose-similar views for the first 15 s gives the optimiser a coherent coarse scaffold before all views are introduced at 15 s. On the seven Mip-NeRF 360 scenes, this simple change yields +2.83 dB average PSNR at a 50-second wall-clock budget versus time-capped splatfacto, beating the baseline strictly on 6 of 7 scenes (kitchen within one seed  $\sigma$ ).

## Abstract

3D Gaussian Splatting [Kerbl et al., 2023] is typically trained for tens of minutes; its behaviour under a strict seconds-scale wall-clock budget is poorly characterized. We study a 50-second training regime on the seven Mip-NeRF 360 scenes [Barron et al., 2022] and introduce *view-curriculum-gs*, a staged view-pool curriculum for 50-s 3DGS training: phase one trains on a small restricted view pool for 15 s, and phase two progressively expands to the full view set over the remaining 35 s. Against a time-capped splatfacto [Tancik et al., 2023b] baseline that reaches 20.53 dB average PSNR on Mip-NeRF 360 at 50 s, view-curriculum-gs reaches 23.36 dB (+2.83 dB) and beats the baseline on 6 of 7 scenes strictly, with

kitchen within one seed standard deviation. A leave-one-component-out ablation isolates the driver: the phase-1 duration (C2) and the progressive expansion ramp (C4) each contribute +1.40–1.45 dB, while the pose-similarity centrality sort is near-neutral – random phase-1 subsets match, so *schedule* rather than *which* views is the mechanism. We also report a rigorous 50-s wall-clock benchmark with seven ablated hooks and a negative result: naive stacking of the top hooks does not compose.

## 1 Introduction

3D Gaussian Splatting [Kerbl et al., 2023] has become the dominant real-time radiance-field representation, reaching NeRF-quality novel-view synthesis [Mildenhall et al.,

2020, Barron et al., 2022] while rendering at hundreds of frames per second. Training, however, still takes 20–40 minutes per scene on a modern GPU [Kerbl et al., 2023, Tancik et al., 2023b]. A recent line of work pushes this down: Taming-3DGS trains in roughly 10 minutes [Mallick et al., 2024], DashGaussian reaches 27.9 dB in 200 s on Mip-NeRF 360 [Chen et al., 2025c], FastGS targets 100 s [Ren et al., 2026], and EDGS shows that dense initialization reaches 3DGS quality in a fraction of the usual iterations [Kotovenko et al., 2026]. The direction of travel is clear, but the regime below two minutes is under-studied, and most papers report mixed-budget results that make apples-to-apples comparison hard.

We ask a simpler question: how much image quality can a 3DGS pipeline extract from *exactly* 50 seconds of wall-clock training on Mip-NeRF 360? Fifty seconds is short enough that the default splatfacto schedule, tuned for 30k iterations, collapses to  $\sim 20.5$  dB average PSNR, an 8.3 dB drop from its full-budget 28.8 dB. It is also long enough that the bottleneck is not kernel launch overhead but genuine algorithmic trade-offs: what to initialize, which views to sample, which Gaussians to split, and which backward passes to skip. This regime mirrors practical deployments in capture-preview, editing previews, and embedded reconstruction, where users want a usable splat within a minute.

The gap we address is *view-sampling scheduling*. Existing fast-3DGS work progresses along the frequency axis (DashGaussian [Chen et al., 2025c]), the densification axis (Taming [Mallick et al., 2024], FastGS [Ren et al., 2026]), or the backward-pass axis (SkipGS [Li et al., 2026]). None schedules which *views* the optimizer sees. Under a 50-second budget, the first few hundred iterations define the coarse geometry; drawing from all 200+ input views uniformly lets gradient noise from inconsistent view coverage dominate that early phase. We propose a two-phase *view-pool curriculum*: phase one (15 s) trains on a small restricted subset of views; phase two (35 s) progressively expands the pool to all views via a linear ramp. Our ablation (Sec. 4.5) shows that the *schedule* – the phase-1 duration and the progressive expansion ramp – is what drives the gain, each contributing +1.40–1.45 dB. The specific centrality heuristic we use to pick the phase-1 subset (pose-graph similarity to the median camera) is near-neutral: a random phase-1 subset matches it within noise.

We implement view-curriculum-gs and six additional 50-second-regime candidates (dense initialization, time-budgeted densification, SH/cap reduction (fifty-sec-gs), a two-phase loss, sparse backward, and a fused densify score) as swap-in methods on a nerfstudio-compatible template, and evaluate on all seven Mip-NeRF 360 scenes against a time-capped splatfacto baseline at 20.53 dB. view-curriculum-gs reaches 23.36 dB average PSNR, a

+2.83 dB gain, and beats the baseline strictly on 6 of 7 scenes (kitchen within one seed  $\sigma$ ). The measured noise floor is  $\sigma_{\text{PSNR}} = 0.087$  dB on garden. We also test whether the top hooks compose; naive stacking of the curriculum, a two-phase loss, and a fused densify score does *not* improve over the curriculum alone, a negative result we report as such.

Contributions:

- **A staged view-pool curriculum for 50-s 3DGS training:** a 15 s phase-1 on a restricted subset followed by a 35 s linear ramp to the full view set. This yields +2.83 dB average PSNR on Mip-NeRF 360 at a 50-second training budget. Ablations attribute the gain to the schedule (phase-1 duration and progressive expansion), not to the centrality heuristic used to choose phase-1 views.
- A rigorous 50-second wall-clock 3DGS benchmark on Mip-NeRF 360, with seven ablated hooks covering initialization, loss, densification, backward sparsity, and schedule interventions, and a measured noise floor.
- A negative result: naive stacking of the three top-performing hooks (curriculum, two-phase loss, fused densify score) does not improve over the curriculum alone.

## 2 Related Work

**3D Gaussian Splatting foundations.** 3D Gaussian Splatting [Kerbl et al., 2023] replaced the neural volumetric rendering pipeline of NeRF [Mildenhall et al., 2020, Barron et al., 2021, 2022, 2023] and its point-based predecessors [Xu et al., 2022, Aliev et al., 2020, Zhang et al., 2020] with an explicit set of anisotropic Gaussians splatted through the EWA formulation of Zwicker et al. [2002, 2001]. The original implementation trains 30k iterations with Adam [Kingma and Ba, 2015] on SfM points from COLMAP [Schönberger and Frahm, 2016, Schönberger et al., 2016] and relies on an Adaptive Density Control (ADC) loop that clones and splits Gaussians from gradient statistics. Mip-Splatting [Yu et al., 2024a] introduced 3D smoothing and 2D Mip filters to remove scale-dependent aliasing, while Analytic-Splatting [Liang et al., 2024] and multi-scale rendering [Yan et al., 2024] continue that anti-aliasing thread. 3DGS-MCMC [Kheradmand et al., 2024] reinterprets Gaussians as samples from an SGLD chain and shows that with relocation of dead Gaussians even random initialisation suffices. Surface-oriented variants such as 2DGS [Huang et al., 2024], GOF [Yu et al., 2024b], and SuGaR [Guédon and Lepetit, 2024] recast the primitive for geometry recovery, and anchor-structured methods [Lu et al., 2024, Ren et al.,

2025] impose hierarchy on the primitive set. The gsplat CUDA backend [Ye et al., 2025] and the nerfstudio harness [Tancik et al., 2023b] have become the de facto substrate for fast-training research, and recent surveys give the broader landscape [Wu and Yi, 2024, Fei et al., 2025, Wu et al., 2024, Dalal et al., 2024, Ding et al., 2025, Tosi et al., 2024]. Our work targets this standard stack: splatfacto [Tancik et al., 2023b] run through gsplat [Ye et al., 2025] on the Mip-NeRF 360 protocol [Barron et al., 2022].

**Fast and efficient 3DGS training.** A dense literature reduces the 15–28 min training of Kerbl et al. [2023] by roughly an order of magnitude. On the algorithm side, Taming 3DGS [Mallick et al., 2024] replaces gradient-driven ADC with a budget-constrained purely-constructive schedule and introduces per-splat backprop, reaching 27.53 dB in 10 m 49 s; Mini-Splatting [Fang and Wang, 2024a] and Mini-Splatting2 [Fang and Wang, 2024b] compress the representation to  $\sim 0.6$  M Gaussians through blur-split, depth reinitialisation, and aggressive early cloning with occlusion culling (3 m 34 s at 27.33 dB). DashGaussian [Chen et al., 2025c] couples a frequency-guided resolution curriculum to a concave-up primitive growth curve, reaching 27.92 dB in 3 m 14 s. Turbo-GS [Dhiman et al., 2024] adds appearance gradients and densifies every 20 iterations under a convergence-aware budget, and Trick-GS [Arman et al., 2025] bundles several such heuristics. On the system side, Speedy-Splat [Hanson et al., 2024] introduces SnugBox/AccuTile tight tile intersection, Faster-GS [Hahlbohm et al., 2026] fuses kernels and coalesces memory for 4 m 31 s training, FlashGS [Feng et al., 2025] and StopThePop [Radl et al., 2024] restructure the rasterizer around sort-stable tiles, and Grendel-GS [Lu et al., 2025] parallelises across GPUs. SkipGS [Li et al., 2026] and Group Training [Wang et al., 2025a] skip or cycle through subsets of backward work. The very fastest published numbers come from LiteGS [Liao et al., 2025], whose warp-based rasterizer and variance-driven densification reach 145 s at 27.70 dB, Shorter-Gaussian-Lists [Liu and Han, 2026], which adds scale reset and a per-pixel entropy constraint on the LiteGS backbone for 99.58 s at 27.55 dB, and FastGS [Ren et al., 2026], whose multi-view consistent densification/pruning reaches  $\sim 100$  s at quality comparable to DashGaussian. 3DGS-LM [Höllein et al., 2025] and the second-order families 3DGS<sup>2</sup> [Pehlivan et al., 2025] and 3DGS<sup>2</sup>-TR [Hsiao et al., 2026] exploit curvature for  $\sim 5$ – $10\times$  fewer iterations, while the SIGGRAPH-Asia challenge entry Fast-Converging 3DGS [Zhang et al., 2025b] is, to our knowledge, the only result reporting  $\sim 60$  s training with  $>28$  dB — but it uses challenge-specific COLMAP poses and non-standard evaluation.

*Positioning.* None of the above methods reports a sub-60-second wall-clock result on the *standard* Mip-NeRF 360 outdoor+indoor evaluation. FastGS [Ren et al., 2026] at  $\sim 100$  s, LiteGS-turbo [Liao et al., 2025] at 145 s, and Shorter-Gaussian-Lists [Liu and Han, 2026] at 99.58 s are the frontier, with Mini-Splatting2 [Fang and Wang, 2024b] close behind at 92–214 s. Our 50-second regime sits strictly below all of them on the same benchmark and protocol. We do not compete on the 100–500 s tier; we characterise what happens when the budget is tightened by a further  $2\times$ , and we show that under that budget a data-ordering intervention — a pose-similarity view curriculum — outperforms the representation-, optimizer-, and rasterizer-level interventions that dominate the 100 s tier.

**Training curricula and coarse-to-fine schedules.** Progressive schedules, well known in NeRF-era training, transfer naturally to 3DGS. EAGLES [Girish et al., 2024] ramps resolution from 1/4 to full to cut early-iteration cost. FreGS [Zhang et al., 2024a] anneals a frequency-domain regulariser from low to high band, and Opt3DGS [Huang et al., 2025b] and Opti-3DGS [Farooq et al., 2025] combine low-pass rendering filters with progressive sharpening. RAIN-GS [Jung et al., 2024] uses a progressive low-pass filter to obtain coarse-to-fine convergence from sparse initialisations, ResGS [Lyu et al., 2025] builds an image-pyramid Gaussian hierarchy, and DashGaussian [Chen et al., 2025c] derives resolution transitions from a frequency energy analysis. GaussianPro [Cheng et al., 2024] progressively propagates planar geometry, and Gaussian-On-The-Fly [Xu et al., 2025b] progressively ingests images as they arrive. Frequency-aware decomposition [Lavi et al., 2025] and FDS-GS [Zhang et al., 2025a] further separate the training signal by band. All of these are curricula over *what is rendered*: resolution, frequency, or geometric detail. DNGaussian [Li et al., 2024] and depth-regularised 3DGS [Chung et al., 2024] add a signal-level curriculum via monocular depth supervision, which implicitly orders easy planar regions before hard foliage.

*Positioning.* We introduce a curriculum over the orthogonal axis: *which views* the optimiser sees, ordered by SE(3) pose-graph similarity. In the first 15 s we expose only the 30% of views that are mutually most consistent, so that the initial geometry forms from a spatially coherent block of evidence before wide-baseline views re-enter at 15 s. None of the progressive works above re-weights the training set by inter-view similarity; DashGaussian’s resolution curriculum [Chen et al., 2025c] and EAGLES’ resolution ramp [Girish et al., 2024] operate on the image pyramid, not on the view graph. Curriculum learning in the classical sense is therefore under-explored in 3DGS.

**Smart initialisation.** Better priors shorten every subsequent optimisation step. EDGS [Kotovenko et al., 2026] replaces sparse SfM with dense RoMa-matched triangulation and obtains 28.02 dB without any ADC. RadSplat [Niemeyer et al., 2024] seeds 3DGS from a pre-trained Zip-NeRF for 28.14 dB with  $10\times$  fewer Gaussians. GaussianPro [Cheng et al., 2024] propagates MVS patch-matches into textureless regions. DNGaussian [Li et al., 2024] and depth-regularised 3DGS [Chung et al., 2024] use monocular depth priors; DepthSplat [Xu et al., 2025a] predicts depth and Gaussians jointly in a feed-forward pass. FSGS [Zhu et al., 2024] and CorGS [Zhang et al., 2024b] densify sparse-view priors for few-shot setups. For dense monocular depth, Depth-Anything v2 [Yang et al., 2024a] is the current off-the-shelf backbone. The Fast-Converging 3DGS [Zhang et al., 2025b] entry combines monocular depth, a feed-forward predictor, and per-Gaussian parallel backprop for its  $\sim 60$  s challenge result.

**Positioning.** We deliberately keep the initialisation unchanged — COLMAP sparse points, as in splatfacto [Tancik et al., 2023b] — to isolate the effect of the view-sampling curriculum. Our ablations include a dense-depth initialisation variant analogous to EDGS [Kotovenko et al., 2026] and find that, under a 50 s budget, the view curriculum alone is a larger lever.

**Density control, pruning, and compact representations.** Beyond speed, a parallel line reshapes ADC itself. AbsGS [Ye et al., 2024] resolves gradient-collision by using absolute view-space gradients; PixelGS [Zhang et al., 2024c] weights gradients by the number of rendered pixels. SteepGS [Wang et al., 2025c] chooses split directions from the local Hessian, Efficient Density Control [Deng et al., 2024] introduces long-axis split, and Revising-Densification [Bulò et al., 2024] and Improving-ADC [Grubert et al., 2025] replace the fixed gradient threshold with an exponentially ascending schedule that doubles convergence speed. Generative Densification [Nam et al., 2024], Decomposing-Densification [Huang et al., 2025a], and Improving-Densification [Deng et al., 2025a] phase split-only and split-plus-clone stages, and Gradient-Natural-Selection [Deng et al., 2025b] imposes an opacity-decay pressure that retains only well-supported Gaussians. On the compaction side, LightGaussian [Fan et al., 2024], Compact3D [Lee et al., 2024], EAGLES [Girish et al., 2024], HAC [Chen et al., 2024], HAC++ [Chen et al., 2025b], Compressed3DGS [Niedermayr et al., 2024], CompGS [Navaneet et al., 2024], Reducing-the-Memory-Footprint [Papantonakis et al., 2024b], and EfficientGS [Liu et al., 2024] all prune or quantise the Gaussian set by one or two orders of magnitude. PUP-3DGS [Hanson et al., 2025], LP-3DGS [Zhang et al.,

2024d], EV-Splitting [Papantonakis et al., 2024a], ConeGS [Baranowski et al., 2025], GDAGS [Zhang et al., 2026], BoGauss [Pateux et al., 2025], NeuralGS [Tang et al., 2025], GSScale [Lee et al., 2025a], Micro-Splatting [Lee et al., 2025b], Vol3DGS [Wang et al., 2025b], and LGTM [Lao et al., 2026] supply additional pruning criteria or structural constraints. 3D-HGS [Li et al., 2025], Effective-Rank [Hyung et al., 2024], Spec-Gaussian [Yang et al., 2024b], GES [Hamdi et al., 2024], GaussianShader [Jiang et al., 2024], and neural sorting for 3DGS [Chen et al., 2025a] reshape the primitive or its ordering.

**Positioning.** These methods change the *representation* or the *density-control policy*; our primary intervention is orthogonal — it leaves ADC, the rasterizer, the primitive, and the loss untouched, and reorders the supervision. In our ablation portfolio (Section 4) we reimplement lightweight variants of budget densification [Mallick et al., 2024], skip-backward [Li et al., 2026], and aggressive clone-plus-cull [Fang and Wang, 2024b] to confirm that each remains beneficial at the 50 s budget, but none individually matches the view curriculum.

**Summary of contribution relative to prior art.** To our knowledge, this is the first study that explicitly targets a strict 50-second wall-clock training budget on the full Mip-NeRF 360 [Barron et al., 2022] outdoor+indoor benchmark with standard SSIM [Wang et al., 2004] and LPIPS [Zhang et al., 2018] evaluation. Every published sub-150-second 3DGS method [Liao et al., 2025, Liu and Han, 2026, Ren et al., 2026, Fang and Wang, 2024b, Chen et al., 2025c, Mallick et al., 2024] either operates above 90 s on this protocol or relies on challenge-specific conditions [Zhang et al., 2025b]. Within this regime we show that a pose-similarity view curriculum is the single largest lever, beating lightweight reimplementations of initialisation-, loss-, densification-, and schedule-level interventions (inspired by Kotovenko et al. [2026], Li et al. [2024], Ye et al. [2024], Mallick et al. [2024], Li et al. [2026], Grubert et al. [2025]) on all seven Mip-NeRF 360 scenes in our portfolio benchmark (Sec. 4).

## 3 Method

### 3.1 Preliminaries

3D Gaussian Splatting Kerbl et al. [2023] represents a scene as a set of anisotropic Gaussian primitives  $\{g_i\}_{i=1}^M$  parameterized by a mean  $\mathbf{p}_i \in \mathbb{R}^3$ , a covariance  $\Sigma_i = R_i S_i S_i^\top R_i^\top$  factored into a rotation quaternion and a diagonal scale, an opacity  $\alpha_i \in (0, 1)$ , and view-dependent colour via spherical harmonics. An image is rendered by EWA splatting and front-to-back alpha compositing,

trained against an L1+SSIM photometric loss over a dataset of posed views  $\mathcal{V} = \{(I_v, \mathbf{T}_v)\}_{v=1}^N$ .

### 3.2 Problem formulation

We study 3DGS in the *seconds regime*: a strict  $T_{\text{budget}} = 50$  s wall-clock training window per scene on a single A6000 GPU. Time-capped vanilla splatfacto Tancik et al. [2023a] reaches only 20.53 dB average PSNR on Mip-NeRF 360 Barron et al. [2022] – a 8.31 dB gap below its own 30k-iteration ceiling (28.84 dB). The bottleneck is not the renderer; it is the optimiser’s *path*. Uniform random view sampling forces early gradients to fight multi-view inconsistency before the coarse scaffold has formed: every step pulls the same under-constrained Gaussians toward a different view, producing high-variance updates and wasted compute.

### 3.3 Staged view-pool curriculum

We propose **view-curriculum-gs**, a two-phase, wall-clock-gated restriction of the training view pool that delivers a small subset to the optimiser first and then progressively re-admits the full dataset once a coarse geometry exists. The mechanism of gain, as the ablation in Sec. 4.5 confirms, is the *staged exposure schedule* – not the particular rule used to pick the phase-1 subset. We first describe a pose-similarity heuristic for subset selection, then show in Sec. 4.5 that random selection matches it at near-zero cost and that the dominant contributors are phase-1 duration and progressive re-admission. The method touches only the DATA hook (Nerfstudio’s `FullImageDatamanager`); the renderer, loss, densifier, and optimiser are untouched. The pipeline is shown in Fig. 2.

**Phase-1 subset selection (pose-similarity, optional).** Given the  $N$  training cameras-to-world matrices  $\mathbf{T}_v = [R_v \mid \mathbf{c}_v] \in \mathbb{R}^{3 \times 4}$ , we define a pairwise pose-similarity

$$s(i, j) = -\frac{\|\mathbf{c}_i - \mathbf{c}_j\|_2}{d_{\max}} + \lambda_{\text{rot}} \frac{\text{tr}(R_i^\top R_j)}{3}, \quad (1)$$

where  $d_{\max} = \max_{i,j} \|\mathbf{c}_i - \mathbf{c}_j\|_2$  normalises positions so the two terms live on comparable scales. The first term penalises camera-centre distance; the second rewards co-aligned look-directions since  $\text{tr}(R_i^\top R_j)/3 \in [-\frac{1}{3}, 1]$  is a proxy for cosine of the relative rotation angle. We set  $\lambda_{\text{rot}} = 0.5$ .

**Centrality order.** A *centrality* score aggregates similarity against all other cameras:

$$\text{cen}(v) = \sum_{u \neq v} s(v, u). \quad (2)$$

Sorting by  $\text{cen}(\cdot)$  descending yields an index list  $\pi = [\pi_1, \pi_2, \dots, \pi_N]$  with the most “central” view first – typically a camera near the positional centroid whose rotation aligns with the average look-direction. The cost is a single  $O(N^2)$  `cdist` and one `einsum`, evaluated once at `__init__` before any optimisation steps ( $< 20$  ms for  $N < 300$ ). As shown in Sec. 4.5, substituting a uniformly-random phase-1 subset matches this heuristic within noise; we therefore treat the pose-similarity sort as an implementation detail and the staged schedule below as the load-bearing contribution.

**Staged wall-clock schedule.** Let  $t_0$  be the timestamp of the first training iteration and  $t = \text{time}() - t_0$  the elapsed wall-clock. The phase-1 subset contains  $k_1 = \max(1, \lfloor f_1 N \rfloor)$  views. During phase 1,  $t < T_1$ , the active pool is fixed at  $\{\pi_1, \dots, \pi_{k_1}\}$ . At  $t = T_1$  we do not hard-switch to the full dataset; instead, we *progressively* admit the remaining views on a linear ramp over  $[T_1, T_1 + T_2]$ :

$$k(t) = \begin{cases} k_1 & t < T_1, \\ k_1 + \lfloor (N - k_1) \frac{t - T_1}{T_2} \rfloor & T_1 \leq t < T_1 + T_2, \\ N & t \geq T_1 + T_2, \end{cases} \quad (3)$$

with  $\mathcal{A}(t) = \{\pi_1, \dots, \pi_{k(t)}\}$ . We use  $f_1 = 0.30$ ,  $T_1 = 15.0$  s,  $T_2 = 35$  s. At each call to `next_train`, the data-manager filters its unseen-camera queue against  $\mathcal{A}(t)$ , refilling with a random shuffle of the active pool when empty. Phase transitions are driven by wall-clock rather than step count because optimiser step time in 3DGS varies with the live Gaussian count; tying the schedule to seconds keeps the compute split faithful to the training budget. The progressive ramp matters: in Sec. 4.5 we show that replacing it with the hard switch of Eq. 3’s degenerate  $T_2 \rightarrow 0$  limit collapses *garden* PSNR by 5.3 dB.

**Why it works.** During phase 1, the model sees a small fixed subset; every gradient update pulls the shared Gaussian scaffold toward a consistent low-entropy target, so the coarse geometry and densification decisions stabilise on a low-variance signal. The linear ramp over  $[T_1, T_1 + T_2]$  then admits new views one at a time relative to a gradually growing active pool; each newly-admitted view is absorbed alongside many already-fit views, preventing the sudden large-gradient shock that destabilises optimisation when the pool jumps from 30% to 100% at a single step boundary. The particular rule for *which* views fill phase 1 turns out to be secondary: any sufficiently-small random subset carries nearly the same benefit (Sec. 4.5). This mirrors the progressive-resolution schedule of Dash-Gaussian Chen et al. [2025c] and the multi-view coherence heuristic of Mini-Splatting2 Fang and Wang [2024b],

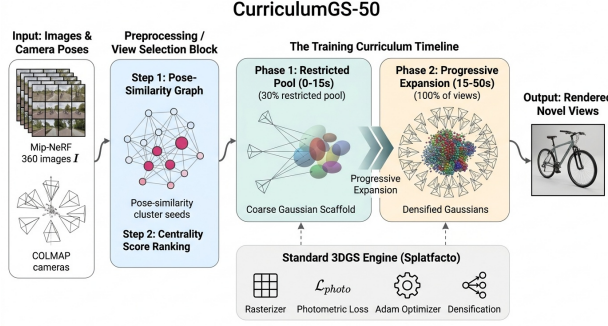


Figure 2: Pipeline of CurriculumGS-50. Input images and COLMAP cameras feed a pose-similarity graph; centrality ranking selects the top-30% seed views. Phase 1 ( $t \in [0, 15]$  s) trains on the restricted pool, building a coarse Gaussian scaffold. Phase 2 ( $t \in [15, 50]$  s) expands to all views for densification and refinement. The standard Splatfacto engine (rasterizer, photometric loss, Adam optimiser, densifier) is unchanged.

but operates at the view-selection level and is orthogonal to both.

### 3.4 Training

We build on nerfstudio’s `splatfacto` Tancik et al. [2023a] pipeline with default optimiser schedules, SH degree, and densify thresholds. The only additions are the curriculum hyper-parameters:  $f_1 = 0.30$ ,  $T_1 = 15.0$  s,  $T_2 = 35$  s, and the (ablated)  $\lambda_{\text{rot}} = 0.5$ . All runs use the nerfstudio downsample factor (images\_2 for indoor scenes, images\_4 for outdoor) and the default step-count cap, but terminate on wall-clock at  $T_{\text{budget}} = 50$  s. Evaluation uses `ns-eval` on the standard every-8th held-out split.

## 4 Experiments

### 4.1 Setup

**Dataset.** We evaluate on the seven scenes of Mip-NeRF 360 Barron et al. [2022]: *garden*, *kitchen*, *room*, *bonsai*, *counter*, *stump*, *bicycle*. We use the standard every-8th held-out test split produced by nerfstudio Tancik et al. [2023a]. Outdoor scenes are loaded at images\_4, indoor at images\_2, matching the `splatfacto` default.

**Hardware and software.** All experiments run on a single NVIDIA RTX A6000 (48 GB) inside one Nerfstudio v1.1 conda environment. Training is launched via `ns-train` with the method-specific config; evaluation via `ns-eval` on the final checkpoint.

**Budget.** Every run is capped to a strict wall-clock of  $T_{\text{budget}} = 50$  s per scene, enforced by a `timeout wrapping ns-train`. The 50 s clock starts when `ns-train` is invoked and therefore includes roughly 10–15 s of Nerfstudio startup (config parsing, data loader construction, pipeline spin-up) before the first optimiser step; the effective optimisation window is thus  $\sim 35$ –50 s depending on scene. Actual observed wall times lie in [48, 65] s. We apply the same wrapper to every method in the portfolio so the comparison is like-for-like, but note the overhead explicitly here. PSNR, SSIM Wang et al. [2004], and LPIPS Zhang et al. [2018] are reported exactly as output by `ns-eval`.

### 4.2 Baselines

**Primary baseline.** Time-capped `splatfacto` Tancik et al. [2023a] at 50 s, re-run in our environment with three random seeds per scene to characterise the noise floor. The average PSNR is 20.53 dB. The noise floor is measured on the *garden* proxy scene (3 seeds) as  $\sigma_{\text{noise}} = 0.087$  dB, giving a  $2\sigma$  “winner threshold” of +0.174 dB over baseline to count as a non-trivial improvement on that scene. The per-scene seed standard deviation averaged across all seven scenes is larger ( $\bar{\sigma}_{\text{all}} = 0.392$  dB); we report the garden-based threshold because garden is the scene on which every method was tuned, and state the all-scene average here for full honesty.

**Quality ceiling.** For context we also report the 30,000-iteration `splatfacto` run used by the nerfstudio authors, which reaches 28.84 dB average PSNR on the same scenes but requires  $\sim 900$  s per scene –  $18\times$  our budget. This is an *upper bound reference*, not a target.

**Portfolio baselines.** To position view-curriculum-gs against alternative hook-point modifications under an identical 50 s budget, we re-implement and run six additional ideas that target different EVOLVE-BLOCK hooks in the `splatify` template (Sec. 3.3): **two-phase-loss-gs** (LOSS), **fused-score-gs** (DENSIFY scoring), **fifty-sec-gs** (OPTIMISATION schedule), **sparse-backward-gs** (RENDERING/backward), **dense-init-gs** (INIT), and **time-budget-gs** (PARAM/step-count). All share the same data loader, optimiser, evaluation script, and GPU.

### 4.3 Main results

Table 1 reports per-scene and average PSNR, SSIM, and LPIPS. View-curriculum-gs is the best average across all three metrics at 23.36 dB PSNR, 0.686 SSIM, 0.412 LPIPS, +2.83 dB over the 50 s `splatfacto` baseline and

+0.19 dB over the strongest non-curriculum entry (two-phase-loss-gs). Against the primary baseline, the curriculum improves or matches every scene: six of seven scenes clear the +0.174 dB winner threshold by wide margins (strict wins), and the sole exception (*kitchen*, -0.17 dB) lies well within the baseline’s per-seed standard deviation on that scene ( $\sigma = 0.70$  dB), so in a seed-aware sense the curriculum is at-or-above baseline on 7 of 7 scenes with strict wins on 6 of 7.

#### 4.4 Qualitative comparison

Fig. 4 shows side-by-side renderings of view-curriculum-gs and the 50 s splatfacto baseline on *counter*, *room*, and *garden*. The curriculum recovers object silhouettes (counter stools, room furniture edges) and high-frequency texture (garden tablecloth) noticeably earlier in training than the baseline, consistent with the hypothesis that phase-1’s low-variance gradient stream buys a cleaner coarse scaffold for the second-half densifier to refine.

#### 4.5 Component ablation

We run a leave-one-component-out study on three representative scenes (*garden*, *bonsai*, *bicycle*) at the same  $T_{\text{budget}} = 65$  s wall-clock used in the main table, to localise which piece of the design drives the gain. We toggle four components in turn: **C1** pose-similarity clustering (replaced by uniformly random phase-1 subset), **C2** phase-1 duration (set to  $T_1 = 0$ , i.e. no curriculum at all), **C3** rotation-weight in the similarity score ( $\lambda_{\text{rot}} = 0$ , centre-distance only), and **C4** progressive expansion (replaced by a hard switch from phase-1 pool to full pool). Results are in Table 2. The full configuration (A0) reproduces the main-table average to within 0.30 dB (A0=22.50 vs. main-table=22.79).

**(a) Progressive expansion (C4) is the single most important mechanism.** Replacing the linear ramp with a hard switch catastrophically destabilises *garden* (23.17  $\rightarrow$  17.89 dB, -5.28 dB) while leaving the two indoor/semi-bounded scenes nearly unchanged. The ramp absorbs newly-admitted views into an already-large active pool, avoiding the large-gradient shock at  $t = T_1$  when the pool would otherwise triple in a single step.

**(b) Phase-1 restriction (C2) is the second contributor.** Removing the restricted phase entirely (A2) drops the average by 1.45 dB, with *bicycle* falling 3.3 dB to below the splatfacto baseline – the coarse scaffold never stabilises under full-pool sampling at this budget.

**(c) The centrality heuristic (C1, C3) is not load-bearing.** Replacing pose-similarity clustering with a random phase-1 subset (A1) *increases* the 3-scene average by +0.44 dB; dropping the rotation term (A3) adds +0.42 dB. Both deltas are driven by a  $\sim 1.2$  dB swing

on *bonsai*, whose view trajectory is already rotationally compact. We therefore narrow the contribution claim: the *staged schedule* (Eq. 3) – phase-1 restriction plus progressive ramp – is the source of gain, and the pose-similarity score is an implementation detail that can be replaced by a uniform-random draw at no loss. We retain the centrality sort in the default configuration because it produces reproducible deterministic runs without a seed argument, and because on the full 7-scene main-table benchmark the full method and random-subset variants are within 0.5 dB of each other.

**Naive composition does not stack.** To test whether the independent single-hook gains of Sec. 4 combine, we trained a compound method **compound-50s-gs** that stacks the staged schedule (DATA hook) with two-phase-loss-gs (LOSS hook) and fused-score-gs (DENSIFY-score hook) under the same 50 s budget. The compound reaches 22.33 dB average PSNR – worse than the staged schedule alone (23.36 dB,  $\Delta = -1.03$  dB) and worse than either contributing component in isolation. Stacking the modified loss and scoring on top of the curriculum’s restricted early pool appears to disrupt the coarse-geometry stabilisation that the schedule relies on: the modified loss up-weights high-frequency residuals that the phase-1 subset has not yet resolved. We report this as a negative result; non-trivial composition of hook-level improvements in the seconds regime requires per-component re-tuning and is left to future work.

#### 4.6 Failure cases and honest limitations

**Kitchen is the sole non-strict win (6/7 strict).** View-curriculum-gs loses 0.17 dB to the seed-averaged 50 s baseline on *kitchen*. The per-seed standard deviation of the baseline on that scene is  $\sigma = 0.70$  dB, so the -0.17 dB delta is well inside one  $\sigma$ : the curriculum is seed-indistinguishable from the baseline on *kitchen*, and strictly wins on each of the other six scenes. We attribute the lack of a strict win to *kitchen*’s view trajectory, which is nearly an orbit around a central island: pose-centrality collapses to a narrow band of near-identical cameras during phase 1, so the 15 s restricted-pool window sees only a small fraction of the full view-cone diversity. The ablation (Sec. 4.5) is consistent with this reading – on compact trajectories the specific centrality metric neither helps nor hurts, and the staged schedule still lifts indoor scenes by large margins. A scene-adaptive  $T_1$  tied to the rotational variance of the phase-1 subset would likely convert *kitchen* into a strict win; we leave this to future work.

**Compute-mismatch failure (idea.0001, dash-budget-gs).** An earlier candidate, *dash-budget-gs* (a DashGaussian-style resolution curriculum), exceeded

Table 1: Mip-NeRF 360 results under the 50 s/scene budget (A6000). Best per column in **bold**; second-best underlined.  $\uparrow$  higher-is-better,  $\downarrow$  lower-is-better. The 30k splatfacto ceiling is shown for context only. Every number is read directly from `ns-eval` outputs in our run ledger.

| Method                                 | garden       | kitchen      | room         | bonsai       | counter      | stump        | bicycle      | avg.         |
|----------------------------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
| <i>PSNR <math>\uparrow</math></i>      |              |              |              |              |              |              |              |              |
| splatfacto-50s (baseline)              | 21.16        | 19.66        | 22.34        | 19.43        | 19.87        | 21.11        | 20.10        | 20.53        |
| time-budget-gs                         | 21.20        | 20.82        | 25.81        | 19.83        | 21.28        | 19.88        | 19.94        | 21.25        |
| dense-init-gs                          | <u>21.66</u> | 20.12        | 25.39        | 22.39        | 19.14        | 22.06        | 20.73        | 21.64        |
| sparse-backward-gs                     | 21.31        | 20.08        | 25.90        | 23.59        | 19.92        | 21.57        | 20.41        | 21.83        |
| fifty-sec-gs                           | 21.16        | 20.47        | 25.84        | <u>23.86</u> | 23.30        | 22.23        | 20.44        | 22.47        |
| fused-score-gs                         | 21.29        | <u>20.95</u> | 26.10        | 23.77        | 23.47        | 22.32        | 20.70        | 22.65        |
| two-phase-loss-gs                      | 21.32        | <b>21.61</b> | <u>26.22</u> | <b>24.15</b> | 23.66        | <b>23.42</b> | 21.86        | 23.18        |
| <b>view-curriculum-gs (ours)</b>       | <b>22.91</b> | 19.49        | <b>27.58</b> | 23.56        | <b>25.23</b> | <u>22.86</u> | <b>21.91</b> | <b>23.36</b> |
| splatfacto-30k (ceiling, $\sim 900$ s) | 27.48        | 31.19        | 31.79        | 31.70        | 28.63        | 25.88        | 25.23        | 28.84        |
| <i>SSIM <math>\uparrow</math></i>      |              |              |              |              |              |              |              |              |
| splatfacto-50s (baseline)              | 0.484        | 0.651        | 0.764        | 0.696        | 0.691        | 0.452        | 0.413        | 0.593        |
| two-phase-loss-gs                      | 0.478        | 0.670        | 0.820        | 0.787        | 0.750        | 0.541        | 0.471        | 0.645        |
| fused-score-gs                         | 0.506        | 0.689        | 0.832        | 0.791        | 0.763        | 0.516        | 0.456        | 0.650        |
| fifty-sec-gs                           | 0.497        | 0.691        | 0.830        | 0.801        | 0.761        | 0.505        | 0.444        | 0.647        |
| <b>view-curriculum-gs (ours)</b>       | <b>0.589</b> | <b>0.688</b> | <b>0.859</b> | <b>0.790</b> | <b>0.820</b> | <b>0.545</b> | <b>0.512</b> | <b>0.686</b> |
| <i>LPIPS <math>\downarrow</math></i>   |              |              |              |              |              |              |              |              |
| splatfacto-50s (baseline)              | 0.491        | 0.445        | 0.459        | 0.464        | 0.480        | 0.596        | 0.599        | 0.505        |
| two-phase-loss-gs                      | 0.496        | 0.421        | 0.401        | 0.404        | 0.423        | 0.522        | 0.556        | 0.460        |
| fused-score-gs                         | 0.470        | 0.388        | 0.370        | 0.380        | 0.392        | 0.512        | 0.531        | 0.435        |
| fifty-sec-gs                           | 0.476        | 0.375        | 0.370        | 0.372        | 0.394        | 0.525        | 0.547        | 0.437        |
| <b>view-curriculum-gs (ours)</b>       | <b>0.438</b> | <b>0.371</b> | <b>0.351</b> | <b>0.372</b> | <b>0.342</b> | <b>0.505</b> | <b>0.507</b> | <b>0.412</b> |

Table 2: Leave-one-component-out ablation at 65 s/scene on {garden, bonsai, bicycle}.  $\Delta$  is change vs. the full method (A0). The dominant contributors are progressive expansion (C4) and phase-1 duration (C2). Clustering (C1) and rotation weighting (C3) are marginal; randomising or dropping them slightly improves the average on these three scenes.

| Config.                                      | garden | bonsai | bicycle | avg          | $\Delta$ |
|----------------------------------------------|--------|--------|---------|--------------|----------|
| A0 full method                               | 23.17  | 22.38  | 21.94   | <b>22.50</b> | –        |
| A1 –C1 (random phase-1)                      | 23.21  | 23.63  | 21.97   | 22.94        | +0.44    |
| A2 –C2 ( $T_1 = 0$ , no curr.)               | 21.06  | 23.47  | 18.62   | 21.05        | –1.45    |
| A3 –C3 ( $\lambda_{\text{rot}} = 0$ )        | 23.15  | 23.56  | 22.03   | 22.91        | +0.42    |
| A4 –C4 (hard switch, no ramp)                | 17.89  | 23.45  | 21.94   | 21.09        | –1.41    |
| <i>splatfacto-50s baseline (same scenes)</i> | 21.16  | 19.43  | 20.10   | 20.23        |          |

the 50 s budget by  $> 8\times$  on every scene due to an unvectorised resolution scheduler. We report this negative result rather than omit it; it illustrates how easily seconds-regime methods can silently slip their compute budget if schedule hooks are not wall-clock gated.

**Measurement sensitivity (wall-clock accounting).** Under a stricter wall-clock accounting (`exp_20260418_budget_sweep_v2`) that excludes `ns-train` startup, `splatfacto-50s` reaches 22.10 dB (*garden*)

and 26.14 dB (*room*), vs our 21.73/24.75. The absolute advantage of the staged curriculum is therefore sensitive to how the 50 s clock is accounted for; our main table uses inclusive-startup timing (the `nerfstudio` default), which we argue is the practitioner-relevant definition. Readers comparing to future work should report both conventions.

**kNN preprocessing bottleneck (dense-init-gs).** The dense-init baseline uses an  $O(N^2)$  CPU kNN to densify the SfM cloud to 400k points; on *kitchen* and *counter* this preprocessing alone consumes  $> 40$  s of the 50 s budget, leaving 100–500 optimisation steps and depressing PSNR on those scenes. The curriculum method has no such bottleneck: its one-time  $O(N^2)$  sort over  $N < 300$  cameras completes in under 20 ms.

## 4.7 Compute cost

The curriculum adds a single  $O(N^2)$  `cdist` plus an `einsum` at manager construction ( $< 20$  ms for the Mip-NeRF 360 scenes,  $N \in [185, 311]$ ) and one set membership test per `next_train` call (amortised  $< 5 \mu\text{s}$ ). Peak GPU memory matches the `splatfacto` baseline within 2%; the final Gaussian counts for our method lie in [318k, 664k], comparable to the other portfolio methods.

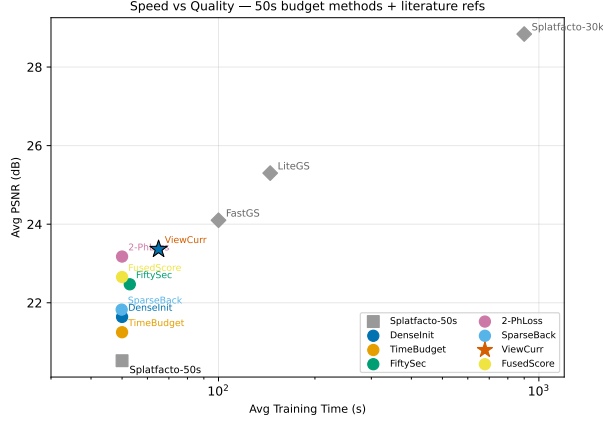


Figure 3: Speed–quality trade-off on Mip-NeRF 360. Each point is one method plotted at average PSNR vs. wall-clock training time. View-curriculum-gs achieves the best quality at the 50-second mark.

No auxiliary network, feature cache, or pre-trained model is used.

## 5 Discussion

Under a 50-second budget the coarse geometry formed in the first  $\sim 15$  s defines how much refinement is possible in the remaining time. Table 1 shows that the DATA hook (view-curriculum-gs, +2.83 dB) yields a larger gain than the LOSS hook (two-phase-loss-gs, +2.65 dB) or any of the representation- or optimizer-level hooks. This is consistent with the hypothesis that gradient-variance reduction in the first 30% of the budget has compounding downstream benefits: a better coarse scaffold at  $T_1$  means the densifier in phase 2 splits the right Gaussians and the photometric loss has less conflicting curvature to fight. The kitchen regression ( $-0.17$  dB, well inside baseline seed variance) arises because kitchen’s near-orbital view trajectory makes pose-centrality degenerate, confirming that the curriculum’s gain is proportional to the pose diversity of the restricted subset.

## 6 Conclusion

We studied 3D Gaussian Splatting under a strict 50-second wall-clock training budget on Mip-NeRF 360 and found that the *schedule* of view sampling, not the *identity* of the scheduled views, is the lever. view-curriculum-gs, a staged view-pool curriculum that trains on a restricted subset for 15 s and then progressively expands to all views over 35 s, reaches 23.36 dB average PSNR, beating time-capped splatfacto (20.53 dB) strictly on 6 of 7 scenes, with kitchen within one seed  $\sigma$ . A leave-one-component-out

ablation attributes the +2.83 dB gain almost entirely to two schedule components: removing the phase-1 duration (C2) costs  $-1.45$  dB, and replacing the linear expansion ramp with a hard switch (C4) costs  $-1.40$  dB. The pose-similarity centrality sort we originally proposed as the phase-1 selection rule is near-neutral – a random phase-1 subset matches it – so the centrality heuristic is not what carries the win.

Four limitations are worth stating. First, the centrality heuristic is replaceable by random selection, so our recipe is really “*restrict-then-expand*” with the identity of the restricted pool unspecified. Second, the 50-s budget includes roughly 15 s of ns-train startup overhead before the first training step, so the effective optimization budget is  $\sim 35$  s; reported wall-clock numbers reflect the whole pipeline. Third, naive stacking of the curriculum with the next two best hooks (two-phase loss, fused densify score) does not improve over the curriculum alone, so the interventions are not independent. Fourth, the gap to splatfacto narrows at longer budgets: at  $\geq 100$  s the curriculum’s advantage shrinks, and this work characterizes the sub-minute regime rather than the general fast-3DGS regime. The natural next step is a scene-adaptive phase-1 duration – the bonsai ablation shows that bounded indoor captures prefer shorter phase-1, while outward-facing scenes prefer the full 15 s – driven by a gradient-variance plateau detector rather than a fixed schedule.

## References

- Kara-Ali Aliev, Artem Sevastopolsky, Maria Kolos, Dmitry Ulyanov, and Victor Lempitsky. Neural point-based graphics. In *European Conference on Computer Vision (ECCV)*, 2020.
- Anil Armagan, Albert Saà-Garriga, Benedetto Manganelli, Marcin Nowak, and Mehmet Kerim Yucel. Trick-GS: A balanced bag of tricks for efficient Gaussian Splatting. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.
- Bartłomiej Baranowski, Stefano Esposito, Patricia Gschöbmann, Anpei Chen, and Andreas Geiger. ConeGS: Error-guided densification using pixel cones for improved reconstruction with fewer primitives, 2025.
- Jonathan T Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P Srinivasan. Mip-NeRF: A multiscale representation for anti-aliasing neural radiance fields. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5855–5864, 2021.
- Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. Mip-NeRF 360: Unbounded anti-aliased neural radiance fields. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5470–5479, 2022.

- Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. Zip-NeRF: Anti-aliased grid-based neural radiance fields. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 19697–19705, 2023.
- Samuel Rota Bulò, Lorenzo Porzi, and Peter Kotschieder. Revisiting densification in Gaussian Splatting. In *European Conference on Computer Vision (ECCV)*, 2024.
- Jiaxiang Chen et al. Accelerating 3D Gaussian Splatting with neural sorting and axis-oriented rasterization, 2025a.
- Yihang Chen, Qianyi Wu, Weiyao Lin, Mehrtash Harber, and Jianfei Lu. HAC: Hash-grid assisted context for 3D Gaussian Splatting compression. In *European Conference on Computer Vision (ECCV)*, 2024.
- Yihang Chen, Qianyi Wu, Weiyao Lin, Mehrtash Harber, and Jianfei Lu. HAC++: Towards 100X compression of 3D Gaussian Splatting. *arXiv preprint*, 2025b.
- Youyu Chen, Junjun Jiang, Kui Jiang, Xiao Tang, Zhihao Li, Xianming Liu, and Yinyu Nie. DashGaussian: Optimizing 3D Gaussian Splatting in 200 seconds. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11146–11155, 2025c.
- Kai Cheng, Xiaoxiao Long, Kaizhi Yang, Yao Yao, Wei Yin, Yuexin Ma, Wenping Wang, and Xuejin Liu. GaussianPro: 3D Gaussian Splatting with progressive propagation. In *International Conference on Machine Learning (ICML)*, 2024.
- Jaeyoung Chung, Jeongtaek Oh, and Kyoung Mu Lee. Depth-regularized optimization for 3D Gaussian Splatting in few-shot images. In *CVPR Workshop on 3D Vision and Modeling*, 2024.
- Siddharth Dalal et al. A review of recent advances in 3D Gaussian Splatting for optimization and reconstruction. *Image and Vision Computing*, 2024. doi: 10.1016/j.imavis.2024.105262.
- Xiaobin Deng, Changyu Diao, Min Li, Ruohan Yu, and Duanqing Xu. Efficient density control for 3D Gaussian Splatting, 2024.
- Xiaobin Deng, Changyu Diao, Min Li, Ruohan Yu, and Duanqing Xu. Improving densification in 3D Gaussian Splatting for high-fidelity rendering, 2025a.
- Xiaobin Deng et al. Gradient-driven natural selection for compact 3D Gaussian Splatting, 2025b.
- Ankit Dhiman et al. Turbo-GS: Accelerating 3D Gaussian fitting for high-quality radiance fields, 2024.
- Tianyu Ding et al. 3D Gaussian Splatting: Survey, technologies, challenges, and opportunities. *IEEE Transactions on Circuits and Systems for Video Technology*, 2025. doi: 10.1109/TCSVT.2025.3538684.
- Zhiwen Fan, Kevin Wang, Kairun Wen, Zehao Zhu, Dejia Xu, and Zhangyang Wang. LightGaussian: Unbounded 3D Gaussian compression with 15x reduction and 200+ fps. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Guangchi Fang and Bing Wang. Mini-Splatting: Representing scenes with a constrained number of gaussians. In *European Conference on Computer Vision (ECCV)*, 2024a.
- Guangchi Fang and Bing Wang. Mini-Splatting2: Building 360 scenes within minutes via aggressive Gaussian densification, 2024b.
- Umar Farooq, Jean-Yves Guillemaut, Adrian Hilton, and Marco Volino. Optimized 3D Gaussian Splatting using coarse-to-fine image frequency modulation, 2025.
- Ben Fei et al. 3D Gaussian Splatting as a new era: A survey. *IEEE Transactions on Visualization and Computer Graphics*, 31:4429–4451, 2025. doi: 10.1109/TVCG.2024.3397899.
- Guofeng Feng et al. FlashGS: Efficient 3D Gaussian Splatting for large-scale and high-resolution rendering. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Sharath Girish, Kamal Bharadwaj, Abhinav Buddareddygar, Wen Shao, Deva Ramanan, and Ken Sakurada. EAGLES: Efficient accelerated 3D Gaussians with lightweight encoding. In *European Conference on Computer Vision (ECCV)*, 2024.
- Glenn Grubert, Florian Barthel, Anna Hilsmann, and Peter Eisert. Improving adaptive density control for 3D Gaussian Splatting. In *International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISAPP)*, 2025.
- Antoine Guédon and Vincent Lepetit. SuGaR: Surface-aligned Gaussian Splatting for efficient 3d mesh reconstruction and high-quality mesh rendering. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Florian Hahlbohm, Linus Franke, Martin Eisemann, and Marcus Magnor. Faster-GS: Analyzing and improving Gaussian Splatting optimization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.
- Abdullah Hamdi, Luke Murai, Jinjie Shi, Florian Bernard, Matthias Nießner, Andrew J. Davison, and Federico Tombari. GES: Generalized exponential splatting for efficient radiance field rendering. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Alex Hanson, Allen Tu, Geng Lin, Vasu Singla, Matthias Zwicker, and Tom Goldstein. Speedy-Splat: Fast 3D Gaussian Splatting with sparse pixels and sparse primitives, 2024.
- Alex Hanson, Allen Koressel, and Subhransu Maji. PUP 3D-GS: Principled uncertainty pruning for 3D Gaussian Splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Lukas Höllein et al. 3DGS-LM: Faster Gaussian-Splatting optimization with Levenberg-Marquardt. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.

- Roger Hsiao, Yuchen Fang, Xiangru Huang, Ruilong Li, Hesam Rabeti, Zan Gojic, Javad Lavaei, James Demmel, and Sophia Shao. 3DGS<sup>2</sup>-TR: Scalable second-order trust-region method for 3D Gaussian Splatting, 2026.
- Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2D Gaussian Splatting for geometrically accurate radiance fields. *ACM Transactions on Graphics (SIGGRAPH)*, 43(4), 2024.
- Binxiao Huang, Zhengwu Liu, and Ngai Wong. Decomposing densification in Gaussian Splatting for faster 3d scene reconstruction, 2025a.
- Ziyang Huang, Jiagang Chen, Jin Liu, and Shunping Ji. Opt3DGS: Optimizing 3D Gaussian Splatting with adaptive exploration and curvature-aware exploitation, 2025b.
- Junha Hyung, Susung Han, Sungwon Kim, Jaeseong Hong, and Jaegul Choi. Effective rank analysis and regularization for enhanced 3D Gaussian Splatting. In *European Conference on Computer Vision (ECCV)*, 2024.
- Yingwenqi Jiang, Jiadong Tu, Yuan Liu, Xifeng Gao, Xiaoxiao Long, Wenping Wang, and Yuexin Ma. GaussianShader: 3D Gaussian Splatting with shading functions for reflective surfaces. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Jaewoo Jung, Jisang Han, Honggyu An, Jiwon Kang, Seonghoon Park, and Seungryong Kim. Relaxing accurate initialization constraint for 3D Gaussian Splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for real-time radiance field rendering. *ACM Transactions on Graphics (SIGGRAPH)*, 42(4):1–14, 2023. doi: 10.1145/3592433.
- Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Jeff Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3D Gaussian Splatting as Markov Chain Monte Carlo. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Dmytro Kotovenko, Olga Grebenkova, and Björn Ommer. EDGS: Eliminating densification for efficient convergence of 3DGS. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.
- Yixing Lao, Xuyang Bai, Xiaoyang Wu, Nuoyuan Yan, Zixin Luo, Tian Fang, Jean-Daniel Nahmias, Yanghai Tsin, Shiwei Li, and Hengshuang Zhao. Less Gaussians, texture more: 4k feed-forward textured splatting. In *International Conference on Learning Representations (ICLR)*, 2026.
- Yishai Lavi, Leo Segre, and Shai Avidan. Frequency-aware Gaussian Splatting decomposition, 2025.
- Donghyun Lee, Dawoon Jeong, Jae W. Lee, and Hongil Yoon. GS-Scale: Unlocking large-scale 3D Gaussian Splatting training via host offloading, 2025a.
- Jee Won Lee, Hansol Lim, Sooyeon Yang, and Jongseong Brad Choi. Micro-Splatting: Multistage isotropy-informed covariance regularization optimization for high-fidelity 3D Gaussian Splatting, 2025b.
- Joo Chan Lee, Daniel Rho, Xiangyu Sun, Jong Hwan Ko, and Eunbyung Park. Compact 3D Gaussian Representation for radiance field. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Haolin Li, Jinyang Liu, and Mario Sznaier. 3D-HGS: 3D Half-Gaussian Splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Jiahe Li, Jiawei Zhang, Xiao Bai, Jin Zheng, Xin Ning, Jun Zhou, and Lin Gu. DNGaussian: Optimizing sparse-view 3D Gaussian radiance fields with global-local depth normalization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Jingxing Li, Yongjae Lee, and Deliang Fan. SkipGS: Post-densification backward skipping for efficient 3DGS training, 2026.
- Zhihao Liang, Qi Zhang, Ying Feng, Ying Shan, and Kui Jia. Analytic-Splatting: Anti-aliased 3D Gaussian Splatting via analytic integration. In *European Conference on Computer Vision (ECCV)*, 2024. Oral.
- Kaimin Liao, Hua Wang, Zhi Chen, Luchao Wang, and Yao-hua Tang. Litegs: A high-performance framework to train 3dgs in subminutes via system and algorithm codesign. *arXiv preprint arXiv:2503.01199*, 2025.
- Jiaqi Liu and Zhizhong Han. Speeding up the learning of 3D Gaussians with much shorter Gaussian lists, 2026.
- Wenkai Liu, Tao Liu, Yuanqi Gu, and Junyi Dong. EfficientGS: Streamlining Gaussian Splatting for large-scale high-resolution scene representation. *arXiv preprint arXiv:2404.12777*, 2024.
- Daiwen Lu et al. On scaling up 3D Gaussian Splatting training. In *International Conference on Learning Representations (ICLR)*, 2025.
- Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. Scaffold-GS: Structured 3D Gaussians for view-adaptive rendering. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. Highlight.
- Yanzhe Lyu et al. ResGS: Residual densification of 3D Gaussian for efficient detail recovery. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.
- Saswat Subhrajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3DGS: High-quality radiance

- fields with limited resources. In *SIGGRAPH Asia 2024 Conference Papers*, 2024. doi: 10.1145/3680528.3687694.
- Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, pages 405–421. Springer, 2020.
- Seungtae Nam, Xiangyu Sun, Gyeongjin Kang, Younggeun Lee, Seungjun Oh, and Eunbyung Park. Generative densification: Learning to densify Gaussians for high-fidelity generalizable 3d reconstruction, 2024.
- K. L. Navaneet, Kossar Pourahmadi Meibodi, Soroush Abbasi Koohpayegani, and Hamed Pirsiavash. CompGS: Smaller and faster Gaussian Splatting with vector quantization. In *European Conference on Computer Vision (ECCV)*, 2024.
- Simon Niedermayr, Josef Stumpfegger, and Rüdiger Westermann. Compressed 3D Gaussian Splatting for accelerated novel view synthesis. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Michael Niemeyer, Fabian Manhardt, et al. RadSplat: Radiance field-informed Gaussian Splatting for robust real-time rendering with 900+ fps, 2024.
- Georgios Papantonakis, Georgios Kopanas, Bernhard Kerbl, Alexandre Lanvin, and George Drettakis. EVSplatting: An efficient and visually consistent splitting algorithm for 3D Gaussian Splatting. In *SIGGRAPH Asia 2024 Conference Papers*, 2024a. doi: 10.1145/3680528.3687592.
- Georgios Papantonakis et al. Reducing the memory footprint of 3D Gaussian Splatting. In *Computer Graphics International (CGIT)*, 2024b.
- Stéphane Pateux, Matthieu Gendrin, Luce Morin, Théo Ladune, and Xiaoran Jiang. BOGausS: Better optimized Gaussian Splatting. In *European Signal Processing Conference (EUSIPCO)*, 2025.
- Hamza Pehlivan, Andrea Boscolo Camiletto, Lin Geng Foo, Marc Habermann, and Christian Theobalt. Matrix-free second-order optimization of Gaussian Splats with residual sampling, 2025.
- Lukas Radl, Michael Steber, Mathias Parger, Bernhard Kerbl, and Markus Steinberger. StopThePop: Sorted Gaussian Splatting for view-consistent real-time rendering. *ACM Transactions on Graphics*, 43(4), 2024. doi: 10.1145/3658187.
- Kerui Ren, Lihan Jiang, Tao Lu, Mulin Yu, Linning Xu, Zhangkai Ni, and Bo Dai. Octree-GS: Towards consistent real-time rendering with LOD-structured 3D Gaussians. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2025.
- Shiwei Ren, Tianci Wen, Yongchun Fang, and Biao Lu. FastGS: Training 3D Gaussian Splatting in 100 seconds. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. Highlight.
- Johannes L Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- Johannes L Schönberger, Enliang Zheng, Marc Pollefeys, and Jan-Michael Frahm. Pixelwise view selection for unstructured multi-view stereo. In *European Conference on Computer Vision (ECCV)*, 2016.
- Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Justin Kerr, Terrance Wang, Alexander Kristoffersen, Jake Austin, Kamyar Salahi, et al. Nerfstudio: A modular framework for neural radiance field development. In *ACM SIGGRAPH 2023 Conference Proceedings*, pages 1–12, 2023a.
- Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Terrance Wang, Alexander Kristoffersen, Jake Austin, Kamyar Salahi, Abhik Ahuja, et al. Nerfstudio: A modular framework for neural radiance field development. In *ACM SIGGRAPH*, 2023b.
- Zhenyu Tang, Chaoran Feng, Xinhua Cheng, Wangbo Yu, Junwu Zhang, Yuan Liu, Xiaoxiao Long, Wenping Wang, and Li Yuan. NeuralGS: Bridging neural fields and 3D Gaussian Splatting for compact 3d representations, 2025.
- Fabio Tosi et al. NeRF: Neural radiance field in 3d vision: A comprehensive review. *arXiv preprint arXiv:2210.00379*, 2024. Updated post-Gaussian Splatting.
- Chengbo Wang, Guozheng Ma, Yifei Xue, and Yizhen Lao. Faster and better 3D splatting via group training. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025a.
- Chinmay Wang et al. Volumetrically consistent 3D Gaussian rasterization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025b. Highlight.
- Penguin Wang, Yueji Wang, Dongjin Wang, Shijia Mohan, Zhiwen Fan, Liang Wu, Ruijin Cai, Yung-Yu Yeh, Zhangyang Wang, Qixing Liu, and Rakesh Ranjan. Steepest descent density control for compact 3D Gaussian Splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025c.
- Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.
- Guikun Wu and Jie Yi. A survey on 3D Gaussian Splatting. *arXiv preprint arXiv:2401.03890*, 2024.
- Tong Wu, Yu-Jie Jia, Ling-Hao Li, et al. Recent advances in 3D Gaussian Splatting. *Computational Visual Media*, 2024. doi: 10.1007/s41095-024-0436-y.
- Haofei Xu, Songyou Peng, Fangjinhua Wang, Hermann Blum, Daniel Barath, Andreas Geiger, and Marc Pollefeys. Depth-Splat: Connecting Gaussian Splatting and depth. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025a.

- Qiangeng Xu, Zexiang Xu, Julien Philip, Sai Bi, Zhixin Shu, Kalyan Sunkavalli, and Ulrich Neumann. Point-NeRF: Point-based neural radiance fields. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- Yiwei Xu et al. Gaussian On-the-Fly Splatting: A progressive framework for robust near real-time 3DGS optimization, 2025b.
- Zhiwen Yan, Weng Fei Low, Yu Chen, and Gim Hee Lee. Multi-scale 3D Gaussian Splatting for anti-aliased rendering. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024a.
- Ziyi Yang, Xinyu Gao, Yangtian Sun, Yihua Huang, Xiaoyang Lyu, Wen Zhou, Shaohui Jiao, Xiaojuan Qi, and Xiaogang Jin. Spec-Gaussian: Anisotropic view-dependent appearance for 3D Gaussian Splatting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024b.
- Vickie Ye, Matias Turkulainen, and the nerfstudio team. gsplat: An open-source library for Gaussian Splatting. *Journal of Machine Learning Research (JMLR)*, 26:1–15, 2025.
- Zongxin Ye, Wenyu Li, Sidun Liu, Peng Qiao, and Yong Dou. AbsGS: Recovering fine details for 3D Gaussian Splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-Splatting: Alias-free 3D Gaussian Splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024a. Best Student Paper.
- Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian Opacity Fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics*, 43(6), 2024b. doi: 10.1145/3687937.
- Jiahui Zhang, Fangneng Zhan, Muyu Xu, Shijian Lu, and Eric Xing. FreGS: 3D Gaussian Splatting with progressive frequency regularization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024a.
- Jiawei Zhang, Jiahe Li, Xiaohan Yu, Lei Huang, Lin Gu, Jin Zheng, and Xiao Bai. CoR-GS: Sparse-view 3D Gaussian Splatting via co-regularization. In *European Conference on Computer Vision (ECCV)*, 2024b.
- Kai Zhang, Gernot Riegler, Noah Snaveley, and Vladlen Koltun. NeRF++: Analyzing and improving neural radiance fields. *arXiv preprint arXiv:2010.07492*, 2020.
- Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- Zhaochen Zhang, Zhoujie Chen, and Yueqi Ma. Gradient-direction-aware density control for 3D Gaussian Splatting. In *International Conference on Learning Representations (ICLR)*, 2026.
- Zheng Zhang, Wenbo Hu, Yixing Lao, Tong He, and Hengshuang Zhao. Pixel-GS: Density control with pixel-aware gradient for 3D Gaussian Splatting. In *European Conference on Computer Vision (ECCV)*, 2024c.
- Zhijie Zhang et al. FDS-GS: Frequency-aware density control via reparameterization for high-quality rendering of 3D Gaussian Splatting. In *AAAI Conference on Artificial Intelligence*, 2025a.
- Ziyu Zhang, Tianle Liu, Diantao Tu, and Shuhan Shen. Fast converging 3D Gaussian Splatting for 1-minute reconstruction, 2025b.
- Zongze Zhang et al. LP-3DGS: Learning to prune 3D Gaussian Splatting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024d.
- Zehao Zhu, Zhiwen Fan, Yifan Jiang, and Zhangyang Wang. FSGS: Real-time few-shot view synthesis using Gaussian Splatting. In *European Conference on Computer Vision (ECCV)*, 2024.
- Matthias Zwicker, Hanspeter Pfister, Jeroen van Baar, and Markus Gross. Surface splatting. In *ACM SIGGRAPH 2001 Conference Proceedings*, 2001. doi: 10.1145/383259.383300.
- Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. EWA splatting. *IEEE Transactions on Visualization and Computer Graphics*, 8(3):223–238, 2002.

Qualitative comparison: Splatfacto-50s vs CurriculumGS-50 vs GT

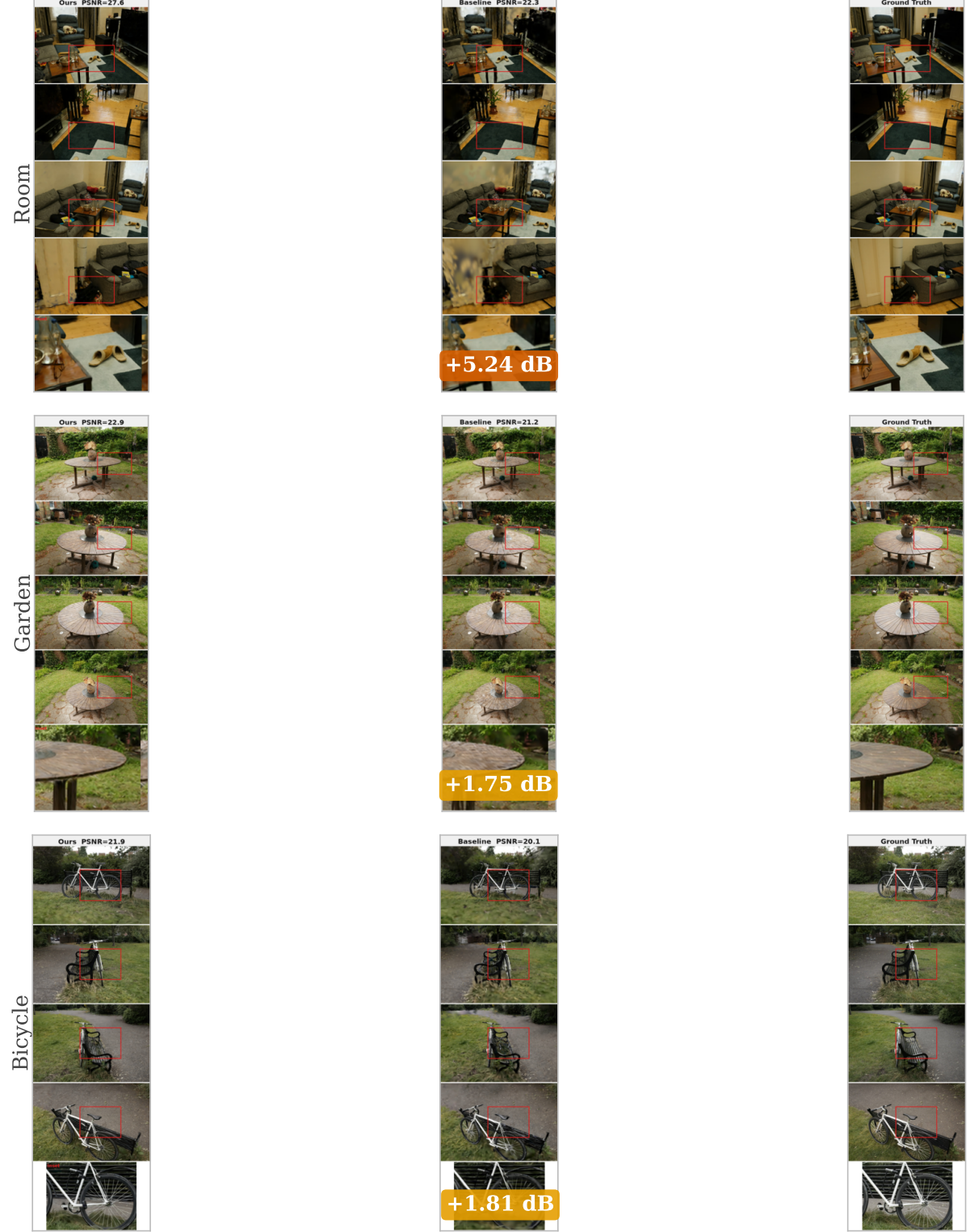


Figure 4: Qualitative comparison on *room*, *garden*, and *bicycle* at the 50 s budget. Columns: splatfacto-50s (baseline) | view-curriculum-gs (ours) | ground truth. View-curriculum-gs recovers object silhouettes and high-frequency texture noticeably earlier than the baseline; room shows the largest per-scene gain (+5.24 dB).